ARCANE ENGINE

A Next.js-Inspired Browser Game Framework

Product Requirements Document

Designed for AI-Assisted Development with Claude Code & OpenAI Codex

Version 0.1 • March 2026

1. Overview

Arcane Engine is a lightweight, convention-over-configuration game framework for the browser. It wraps Three.js with an Entity Component System (ECS) architecture and file-based conventions, giving developers (and AI coding agents) a fast, predictable way to build 3D browser games.

Think of it as what Next.js did for React, but for game development: sensible defaults, a clear folder structure, and a dev loop that just works out of the box.

1.1 Why This Exists

Browser game development today is fragmented. You either export from Unity/Unreal to WebGL (heavy, opaque builds) or wire together Three.js, a physics library, an ECS, and an asset pipeline from scratch every time. There is no framework that provides sensible defaults with escape hatches.

1.2 Design Principles

- **Convention over configuration.** The file system IS the architecture. Drop a file in the right folder and it works.
- **Agent-first API design.** Small API surface, repetitive patterns, typed plain objects. An AI agent should be able to add a feature by creating one file.
- **Escape hatches everywhere.** Every default (renderer, physics, input) is swappable. The framework is opinionated but not locked down.
- **Minimal abstraction.** Thin wrappers, not rewrites. Three.js is still Three.js underneath.

---

2. Key Concepts (Glossary)

These terms are used throughout this document and the codebase. Understanding them is essential.

Concept	What It Is	Analogy
Scene	A self-contained game state: menu, level, game-over screen	A page/route in Next.js
Entity	Any object in the game: player, enemy, tree, bullet	A DOM element on a webpage
Component	A plain data object attached to an entity: position, health, mesh	A CSS class or HTML attribute
System	A function that runs every frame on matching entities	An event handler or middleware

World	The container that stores all entities and their components	The global state/store
Game Loop	Runs 60x/sec: read input, run systems, render	requestAnimationFrame cycle

---

3. Agent Tool Profiles

This project is designed to be built using two AI coding agents in parallel. Understanding their strengths shapes how work is divided.

3.1 Claude Code

- Agentic CLI tool that reads your codebase, edits files, runs commands, and runs tests
- Uses CLAUDE.md for persistent project instructions and context
- Supports sub-agents for parallel subtasks within one session
- Strong at: architecture decisions, multi-file refactors, iterative debugging, writing tests
- Best for: foundational work, core engine code, integration, documentation

3.2 OpenAI Codex

- Cloud-based agent that runs tasks in isolated sandbox environments preloaded with your repo
- Uses AGENTS.md for project instructions and conventions
- Can run many tasks in parallel, each in its own sandbox
- Strong at: well-scoped feature implementation, parallel independent tasks, PR-ready patches
- Best for: building individual systems, components, examples, and tests as parallel tasks

3.3 Shared Configuration

Both tools read markdown instruction files from the repo root. We maintain both CLAUDE.md and AGENTS.md with identical content so either agent understands the project conventions, folder structure, and coding standards.

---

4. Architecture

4.1 Folder Structure

```
arcane-engine/
├── packages/
│   ├── core/              # ECS, game loop, world
```

```
│   ├── renderer/          # Three.js wrapper, render system
│   ├── input/             # Keyboard, mouse, gamepad
│   └── create-arcane/     # CLI scaffolder (npx create-arcane)
├── templates/
│   └── starter/           # Default project template
│       ├── game.config.ts
│       ├── scenes/
│       │   └── main.ts
│       ├── components/
│       ├── systems/
│       └── assets/
├── examples/
│   └── hello-cube/        # MVP demo
├── CLAUDE.md
├── AGENTS.md
└── package.json
```

4.2 Technology Stack

Layer	Technology	Why
Language	TypeScript (strict)	Type safety helps agents and humans alike
Renderer	Three.js	Most mature browser 3D library, huge ecosystem
Build Tool	Vite	Fast HMR, zero config for TS, standard tooling
Package Manager	pnpm	Fast, disk-efficient, great monorepo support
Monorepo	pnpm workspaces	Simple, no extra tooling needed
Testing	Vitest	Fast, Vite-native, TypeScript-first

4.3 Data Flow (Per Frame)

```
1. Input System     →  reads keyboard/mouse state into components
2. User Systems     →  movement, AI, game logic (run in registration order)
3. Render System    →  syncs entity positions/rotations to Three.js objects
4. Three.js         →  draws the frame
```

---

5. Staged Build Plan

The project is divided into 6 stages. Each stage produces a working, testable artifact. Stages are designed so that Codex can parallelize tasks within each stage while Claude Code handles integration and architecture.

Stage 1: Project Skeleton + ECS Core

Goal: A working monorepo with a tested ECS that can create entities, add/remove components, and query by component type.

Time estimate: 2–3 hours

Agent: Claude Code (solo — this is foundational, needs coherent architecture)

Deliverables

- pnpm monorepo with packages/core initialized
- World class: createEntity(), addComponent(), removeComponent(), getComponent()
- Query system: query(world, [Position, Velocity]) returns matching entity IDs
- System registry: registerSystem(world, fn), runSystems(world, deltaTime)
- Full Vitest test suite for all ECS operations
- CLAUDE.md and AGENTS.md with project conventions

Key Interfaces

```
type Entity = number;
type ComponentType<T> = { name: string; default: () => T };

interface World {
  entities: Set<Entity>;
  components: Map<string, Map<Entity, any>>;
  systems: SystemFn[];
  nextId: number;
}

type SystemFn = (world: World, dt: number) => void;
```

Acceptance Criteria

- Can create 1,000 entities with 3 components each in under 10ms
- All CRUD operations on components are tested
- Query correctly filters entities by component combination

Stage 2: Game Loop + Three.js Renderer

Goal: A spinning cube on screen, driven entirely by the ECS. The game loop runs at 60fps with a fixed timestep.

Time estimate: 2–3 hours

Agent: Claude Code for game loop + integration; Codex for renderer package (parallel)

Deliverables

- packages/renderer: creates canvas, camera, scene, WebGLRenderer automatically
- Game loop with fixed timestep (16.67ms ticks) and interpolation
- RenderSystem: syncs Position and Rotation components to Three.js Object3D transforms
- MeshComponent: wraps a Three.js Mesh, auto-added to scene on creation
- Working example: a colored cube that rotates via a SpinSystem

Acceptance Criteria

- Canvas appears fullscreen with no manual setup
- Cube rotates smoothly at 60fps
- Adding a new entity with Position + Mesh makes it appear on screen automatically

Stage 3: Input System + Player Control

Goal: WASD moves an entity. Mouse rotates camera. Input state is readable as components.

Time estimate: 2–3 hours

Agent: Codex (well-scoped, independent from renderer internals)

Deliverables

- packages/input: keyboard and mouse state tracking
- InputState component: { keys: Set<string>, mouse: { x, y, dx, dy } }
- Controllable component: tag that marks an entity as player-controlled
- MovementSystem: reads InputState + Controllable, updates Position
- CameraFollowSystem: camera tracks the controllable entity

Acceptance Criteria

- WASD moves the player cube in world space
- Mouse movement rotates the camera around the player
- Input state is queryable as a regular component (testable, debuggable)

Stage 4: Scene System + File Conventions

Goal: Scenes are loaded from the scenes/ folder. Each scene file exports a setup function. Scene switching works.

Time estimate: 3–4 hours

Agent: Claude Code (architecture-sensitive: Vite integration, dynamic imports, conventions)

Deliverables

- Scene interface: { setup(world): void; teardown?(world): void }
- SceneManager: loadScene(name), handles teardown of previous scene

- Vite plugin or convention that auto-discovers scenes/*.ts
- game.config.ts: initial scene, renderer settings, canvas options
- Two example scenes with a transition between them

Acceptance Criteria

- Creating scenes/level2.ts and exporting a setup function makes it loadable by name
- Switching scenes clears all entities and systems, then runs the new setup
- game.config.ts is the single entry point for project settings

Stage 5: CLI Scaffolder (create-arcane)

Goal: npx create-arcane my-game produces a working project from the starter template.

Time estimate: 1–2 hours

Agent: Codex (self-contained, template copying + string replacement)

Deliverables

- packages/create-arcane: CLI that copies templates/starter into a new directory
- Replaces placeholder names with the project name
- Runs pnpm install and opens dev server
- README.md with getting-started instructions

Acceptance Criteria

- npx create-arcane my-game && cd my-game && pnpm dev shows the demo scene
- Generated project has correct package.json with arcane dependencies

Stage 6: Hello Cube Demo + Documentation

Goal: A polished demo: player cube, floating cubes, scene switching, plus documentation good enough for someone to start building.

Time estimate: 2–3 hours

Agent: Codex for demo content; Claude Code for documentation and final integration

Deliverables

- examples/hello-cube: player-controlled cube, 20 floating cubes with random velocities, ground plane, basic lighting
- Two scenes: gameplay scene + title screen with 'Press Enter to Start'
- README.md: installation, folder conventions, how to add an entity, how to add a system, how to add a scene
- API reference: all public types and functions documented with JSDoc
- CONTRIBUTING.md with conventions for AI agents

Acceptance Criteria

- A new developer (or agent) can read the README, scaffold a project, and add a new entity type within 10 minutes
- All packages have passing tests
- The demo runs at 60fps with 100 entities on a mid-range laptop

---

6. Agent Workflow Strategy

This section describes how to coordinate Claude Code and Codex for maximum velocity.

6.1 Task Division Pattern

Task Type	Assign To	Reason
Architecture decisions	Claude Code	Needs full codebase context, iterative
Core engine (ECS, loop)	Claude Code	Foundational, everything depends on it
Independent packages	Codex	Self-contained, can run in parallel sandbox
Individual systems	Codex	Well-scoped: one file, one function, clear interface
Integration + wiring	Claude Code	Multi-file, needs to see how pieces connect
Tests for isolated units	Codex	Parallel, independent, clear pass/fail
Documentation	Both	Codex for API docs, Claude Code for guides

6.2 Parallel Execution Windows

Within each stage, look for tasks that can run simultaneously on Codex:

- **Stage 2:** Codex builds renderer package WHILE Claude Code builds game loop
- **Stage 3:** Codex builds entire input package independently
- **Stage 5+6:** Codex builds CLI scaffolder AND demo content in parallel

6.3 Instruction Files

Both CLAUDE.md and AGENTS.md should contain:

- Project structure and package responsibilities
- Coding conventions: strict TypeScript, no classes for components, systems are pure functions

- Testing requirements: every public function has a Vitest test
- Naming conventions: PascalCase for component types, camelCase for systems
- Commit format: conventional commits (feat:, fix:, docs:, etc.)

---

7. Post-MVP Roadmap (Out of Scope for Now)

These features are intentionally excluded from the MVP but represent the natural next steps:

- **Physics:** Rapier WASM integration as a swappable physics system
- **Asset pipeline:** GLTF/GLB loader with auto-component generation
- **Audio:** Web Audio API wrapper as a system
- **Collision detection:** Spatial hashing or simple AABB checks
- **UI overlay:** HTML/CSS overlay system for HUD, menus, dialogs
- **Networking:** WebSocket state sync for multiplayer
- **WebGPU renderer:** Swap Three.js renderer for WebGPU when ready
- **Plugin system:** Third-party systems/components installable via npm

---

8. Initial Seed Prompt

Copy the prompt below into Claude Code to kick off Stage 1. This gives the agent full context to begin building.

SEED PROMPT — Paste into Claude Code

```
I'm building Arcane Engine — a lightweight, Next.js-inspired game framework
for the browser. It wraps Three.js with an ECS architecture and
file-based conventions.

## What I need you to do right now (Stage 1):

1. Initialize a pnpm monorepo at the repo root
2. Create packages/core with TypeScript + Vitest
3. Implement the ECS in packages/core/src/:
   - world.ts: World creation, entity CRUD
   - component.ts: defineComponent() factory, typed component storage
   - query.ts: query(world, [...componentTypes]) returns entity IDs
   - system.ts: registerSystem(), runSystems(world, deltaTime)
4. Write comprehensive Vitest tests in packages/core/tests/
5. Create CLAUDE.md and AGENTS.md at root with:
   - Project overview and architecture
   - Folder structure and package responsibilities
   - Coding conventions: strict TS, no classes for components,
```

```
     systems are (world: World, dt: number) => void
   - All components are plain typed objects created by defineComponent()
   - Naming: PascalCase component types, camelCase system functions
   - Testing: every public function gets a Vitest test
   - Commit format: conventional commits

## Design constraints:
- Entity is just a number (ID). No entity class.
- Components are plain objects stored in Map<Entity, T>.
- defineComponent<T>(name, defaultFn) returns a ComponentType<T>
- Systems are pure functions: (world: World, dt: number) => void
- Query must be fast: use bitmask or set intersection
- Target: create 1,000 entities with 3 components each in <10ms

## Tech stack:
- TypeScript (strict mode)
- pnpm workspaces for monorepo
- Vitest for testing
- Vite for building (configure but don't need renderer yet)

Start with the monorepo setup, then build the ECS, then write tests.
After everything passes, create the CLAUDE.md and AGENTS.md files.
```

After Stage 1 is complete and tests pass, open Codex and send it Stage 2 tasks (renderer package) while Claude Code continues with game loop integration.

---

9. Success Metrics

Metric	Target	How to Measure
Time to first render	< 15 minutes from npx create-arcane	Manual test
Add new entity type	1 file, < 20 lines	Count files and lines touched
Add new system	1 file, < 30 lines	Count files and lines touched
Agent scaffold speed	Agent adds a feature in 1 prompt	Test with Claude Code and Codex
Frame rate	60fps with 100 entities	Chrome DevTools performance tab
Bundle size	< 200KB gzipped (framework only)	Vite build output
Test coverage	> 80% on packages/core	Vitest coverage report